Exploiting Cross-Site Scripting to Steal Cookies

1. Overview

Field	Details
Vulnerability	Cross-Site Scripting (XSS)
Category	Client-Side Attack
Attack Objective	Cookie Exposure
Injection Type	JavaScript Injection
Severity	High, depending on cookie sensitivity
Lab Platform	PortSwigger Web Security Academy
Vulnerability Type	XSS Exploitation

2. Description

Cross-Site Scripting can become particularly dangerous when an attacker uses JavaScript execution to access sensitive information stored in browser cookies.

If an application contains an exploitable XSS vulnerability and its authentication cookies are accessible to JavaScript, injected code may read those cookies through the browser's `document.cookie` property.

An attacker may attempt to use the exposed cookie values to compromise a victim's authenticated session.

However, cookies configured with the `HttpOnly` attribute cannot be accessed through ordinary JavaScript, which helps reduce this particular risk.

3. Root Cause

The primary weakness is an existing XSS vulnerability combined with insufficient protection of sensitive cookies.

Potential contributing factors include:

- Missing or incorrectly configured `HttpOnly` cookie attributes.
- Inadequate output encoding.
- Unsafe handling of user-controlled input.
- Sensitive session information exposed to client-side scripts.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify the XSS vulnerability.
3. Determine whether JavaScript execution is possible.
4. Inspect the application's cookie configuration using browser developer tools.
5. Check whether sensitive cookies have the `HttpOnly` attribute.
6. In the authorized lab, use a harmless JavaScript test to inspect which cookies are accessible to client-side scripts.
7. Verify the security implications without transmitting real session credentials to an external destination.

Example JavaScript inspection:

```
None
console.log(document.cookie);
```

This demonstrates which non-HttpOnly cookies are exposed to JavaScript. It does not reveal HttpOnly cookies.

5. Security Impact

Successful exploitation may result in:

- Exposure of JavaScript-accessible cookies.
- Potential session hijacking if an exposed cookie contains a valid session identifier.
- Unauthorized access to a victim's account.
- Account compromise and unauthorized actions.
- Exposure of sensitive application information.

The impact depends on cookie configuration, session management, and additional security controls.

6. Recommended Remediation

- Set the `HttpOnly` attribute on sensitive session cookies.
- Set the `Secure` attribute to restrict cookie transmission to HTTPS.
- Configure an appropriate `SameSite` policy.
- Prevent XSS through context-aware output encoding.
- Avoid unsafe DOM manipulation.
- Use secure JavaScript frameworks and templating systems.
- Implement Content Security Policy (CSP) as an additional defense.
- Rotate session identifiers after authentication and security-sensitive events.
- Invalidate sessions when compromise is suspected.

7. Verification

After remediation:

1. Inspect authentication cookies.
2. Confirm sensitive cookies have the `HttpOnly` and `Secure` attributes.
3. Verify that JavaScript cannot access HttpOnly cookies.
4. Retest the original XSS vulnerability.
5. Confirm that the application no longer exposes session information through injected scripts.

8. References

- PortSwigger: Cross-Site Scripting
- PortSwigger: Exploiting XSS
- MDN: Document.cookie
- OWASP: Session Management Cheat Sheet
- OWASP: XSS Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security assessments. Never collect, transmit, or reuse another person's authentication cookies without explicit authorization.